Palm OS Programming Recipes

Modal Dialogs

You use modal dialogs when you want a dialog to display on top of a form. Common modal dialogs are the "Details" dialogs seen in the built-in applications and the Alarm dialogs seen in DateBook. Occasionally, you might use a modal dialog for a complex alert dialog, but simple alert dialogs tend to be created with an Alert resource and displayed using `FrmAlert`.

To create a "Details" dialog in Constructor:

1. Create a new Form object and name it "Details." (Modal dialogs are really just a special type of form.)
2. Set the left origin to 2 and the width to 156. That will be full-width and leave just enough room for the two-pixel border drawn outside the form bounds.
3. Set the top origin and the height such that their sum equals 158. This aligns the modal dialog with the bottom of the window. For this example, use a top origin of 98 and a height of 60. (A full-height modal dialog has a top origin of 2 and a height of 156.)
4. Turn on "Usable", "Modal", and "Save Behind".
5. If appropriate, create a Help resource for this dialog by setting a Help ID (by convention, this is equal to the form's ID) and clicking the "Create" button. Doing so causes an "i" button to be added to the title bar. The text you enter is displayed when the user taps this button.
6. Set the form's title to "Details."
7. Add the "OK" and "Cancel" command buttons. The "OK" button should have its left coordinate 5, top 43, width 36 and height 12. The "Cancel" button should have its left coordinate 47 and all other coordinates the same as the "OK" button. These coordinates are standard. They leave 4 pixels between the form border and the first button and 4 pixels between each button. If you make the dialog taller or shorter, keep the same distance from the buttons to the bottom edge by adding the top coordinate to the height and verifying that the result is 5 less than the height of the dialog. (5 = the 1-pixel button border, the 2-pixel form border, plus the 2 blank pixels.)
8. Set the labels and object identifiers for the buttons to "OK" and "Cancel".
9. Select the form again. Set the form's Default Button ID to the "OK" button's ID. If the user switches to another application while this dialog is active, the system simulates a press of the default button. You should always choose the action that results in the least possible harm to the user's data. Typically, this would be the action performed by the "OK" or "Done" button, but if the dialog deletes data, the "Cancel" button is more appropriate.
10. Now, add any other necessary controls to this dialog.
11. Open the resource for the main form. Add a button labeled "Details" that the user can tap to display the dialog you just created.

Now you need to decide how to implement the code for the dialog. There are three possible implementations:

- Use `FrmDoDialog` without an event handler. This is the simplest to implement, but offers the least amount of flexibility.
- Use `FrmDoDialog` with a "partial" event handler.
- Use your application's normal event loop and a "full" event handler. This method allows the greatest flexibility but is the most complex to implement.

Option A: FrmDoDialog

If you choose the simplest case, `FrmDoDialog` without an event handler, add the following code to the event handler of the form that opens the Details dialog. Ours is opened from a Details button on the main form:

```
if (eventP->data.ctlSelect.controlID == MainDetailsButton) {
       frmP = FrmInitForm(DetailsForm);

       /* initialize your controls on the form here */

       // open the dialog, and wait until a button is pressed to close it.
       whichButton = FrmDoDialog(frmP);

       if (whichButton == DetailsOKButton) {
          /* get data from controls on the form to save/apply changes */
       }
       FrmDeleteForm(frmP);
       handled=true;
    }
```

You now have a functioning Details dialog. All that remains is to initialize the dialog's controls right before the call to `FrmDoDialog` and to save or apply the changes when the call returns, where the comments indicate.

Option B: FrmDoDialog With an Event Handler

If you want more control over the event processing, you can still use `FrmDoDialog` in conjunction with a "partial" event handler. This handler will let you take care of the events you care about, but still relies on `FrmDoDialog` to do most of the work. `FrmDoDialog` handles any event that this handler doesn't. With this method, use the same code as the previous `FrmDoDialog` example, but insert one line before calling `FrmDoDialog`:

```
FrmSetEventHandler(frmP,  DetailsPartialEventHandler);
```

`FrmDoDialog` calls `DetailsPartialEventHandler` before processing each event, giving your event handler a chance to process each event before the system does. The code for `DetailsPartialEventHandler` could be as follows, if you had a routine to validate the data entered in the Details dialog:

```
static Boolean DetailsPartialEventHandler(EventPtr eventP)
    {
       Boolean handled = false;

       if ((eventP->eType == ctlSelectEvent) &&
           (eventP->data.ctlSelect.controlID == DetailsOKButton)) {
          if (ValidationFailsOnTheInput()) {
             FrmAlert(PleaseFixYourInputAlert);
             // tell FrmDoDialog we handled this event, so it
             // shouldn't do anything
             handled=true;
          }
       }
       return handled;
```

```
    }
```

`FrmDoDialog` runs in its own event loop instead of yours, giving you slightly less control over event handling than you normally have. If you are doing communications or are using event timeouts for some background processing or updating, `FrmDoDialog` is not a good implementation for your dialog. The benefit of `FrmDoDialog` is that you need to write less code and that code is located in only one place in your application.

Option C: Using a Full Event Handler

For the maximum control over the entire event handling of the dialog, you can treat this dialog just like a normal form in your application. Create its event handler function as follows. Note that when the dialog is done, it uses `FrmReturnToForm(0)` to close itself and go back to the previously active form. Therefore the code to save the data is in the dialog's event handler, instead of in the code for the main form.

```
static Boolean DetailsHandleEvent(EventPtr eventP)
    {
       Boolean handled = false;
       FormPtr frmP;

       switch (eventP->eType)  {
          case frmOpenEvent:
             frmP = FrmGetActiveForm();
             /* initialize UI objects here */
             FrmDrawForm(frmP);
             handled = true;
             break;

          case ctlSelectEvent:
             if (eventP->data.ctlSelect.controlID == DetailsOKButton) {
                /* save or apply the changes now */
                FrmReturnToForm(0);
                handled = true;
                break;
             }
             else if (eventP->data.ctlSelect.controlID ==
                         DetailsCancelButton) {
                /* don't save changes, just return. */
                FrmReturnToForm(0);
                handled = true;
                break;
             }

          default:
             break;
       }

       return handled;
    }
```

You also need to specify the event handler for the form, in your `AppHandleEvent` routine:

```
   case DetailsForm:
```

```
        FrmSetEventHandler(frmP, DetailsHandleEvent);
        break;
```

Finally, to display this dialog, your button handler uses `FrmPopupForm` instead of `FrmDoDialog`. The form is loaded by your `AppHandleEvent` routine and deleted when `DetailsHandleEvent` calls `FrmReturnToForm`.

```
   if (eventP->data.ctlSelect.controlID == MainDetailsButton) {
       MenuEraseStatus(0);  // in case the status is still active
       // Some apps might want to save the current focus here.
       FrmPopupForm(DetailsForm);
       handled=true;
    }
```

Now you have a dialog with full control over every event. This code does essentially the same thing as `FrmDoDialog`, but it is integrated into the event handling of your application without a nested event loop. Thus, it is more flexible.